



MARMARA UNIVERSITY FACULTY OF ENGINEERING

EE4065
Introduction to Embedded Image Processing

HOMEWORK 6

Submitted to: Prof. Dr. CEM ÜNSALAN

Due Date: 15.01.2026

	Dept.	STUDENT ID	NAME/SURNAME
1	EE	150720049	RUMEYSA ŞENOL
2	EE	150720996	FATİME EMİN

1. INTRODUCTION

This homework investigates the implementation of Deep Learning-based computer vision on resource-constrained embedded platforms, specifically the STM32F446RE (Nucleo). The primary objective is to realize a Handwritten Digit Recognition system using Convolutional Neural Networks (CNN) trained on the MNIST dataset, as described in Application 13.7 of the reference textbook.

Deploying Deep Neural Networks (DNNs) on microcontrollers (Edge AI) presents significant challenges due to strict limitations in SRAM (128 KB) and Flash memory (512 KB). Unlike desktop environments, embedded inference requires careful management of memory allocation and computational complexity (FLOPs). In this study, we explored a variety of architectures ranging from custom lightweight CNNs to advanced models like Squeeze Net, MobileNetV2, Shuffle Net, and Efficient Net, evaluating their trade-offs between accuracy, model size, and hardware feasibility.

2. METHODOLOGY

The project workflow was executed in three distinct phases:

2.1. Data Preprocessing & Model Training

The MNIST dataset consists of grayscale images. However, modern CNN architectures (e.g., MobileNet, ShuffleNet) typically require input tensors with a minimum spatial resolution of 28 and 3 color channels.

- Resizing: All inputs were upscaled from 28 to 32 using bilinear interpolation.
- Channel Expansion: Grayscale images were converted to RGB format (3 channels) to be compatible with the input layers of transfer-learning models.
- Training: Models were trained using the TensorFlow/Keras framework in a Python environment.

2.2. Quantization & Code Generation

The trained models were exported to TensorFlow Lite (.tflite) format. The ST Edge AI (X-CUBE-AI) middleware was then used to parse these files and generate optimized C code for the Arm Cortex-M4 processor.

- RAM Optimization: To fit the activation buffers into 128 KB SRAM, the "Optimize for RAM" strategy was applied in X-CUBE-AI.
- Static Memory Management: Instead of dynamic allocation (malloc), input and output buffers were managed manually using a Zero-Copy approach in main.c, linking the inference engine directly to static C-arrays stored in the program memory.

3. IMPLEMENTED ARCHITECTURES AND THEORY

To evaluate different design philosophies for embedded vision, the following architectures were implemented:

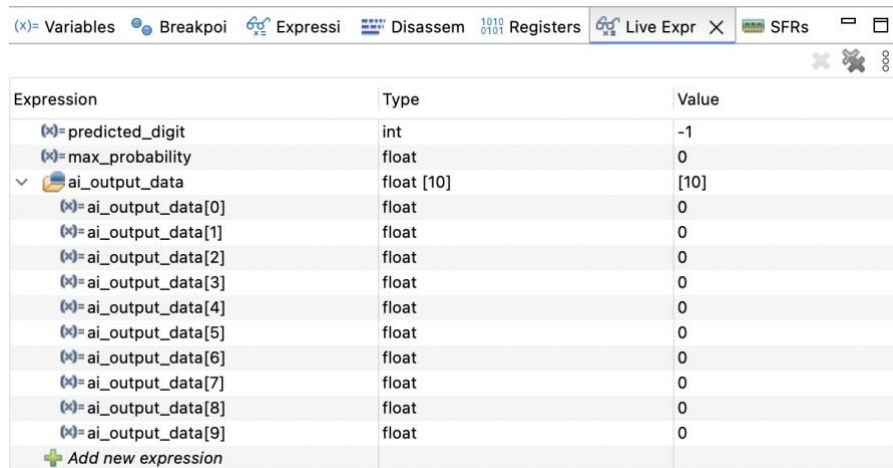
- Custom CNN: A baseline architecture consisting of two Convolutional layers followed by MaxPooling and Dense layers. It is specifically tailored for the input size, offering the smallest footprint.
- SqueezeNet: Designed to reduce parameter count using "Fire Modules", which compress the input channels with filters before expanding them with filters. This architecture is highly suitable for embedded devices due to its compact model size.
- MobileNetV2 (Alpha 0.35): Utilizes "Inverted Residuals" and "Linear Bottlenecks" to preserve information flow. We applied a width multiplier () to aggressively reduce the number of filters per layer, ensuring the model size remained within the 512 KB Flash limit of the STM32F446RE.
- ShuffleNet: Employs "Pointwise Group Convolutions" and "Channel Shuffle" operations to reduce computational cost (FLOPs) while maintaining accuracy. This architecture is particularly effective for reducing inference latency on CPUs.
- EfficientNetB0: Utilizes a "Compound Scaling" method that uniformly scales network width, depth, and resolution. Although it achieves state-of-the-art accuracy, the resulting parameter count and memory requirement exceeded the available Flash memory of the target hardware, highlighting the upper bound of the F446RE board.

4. PROCESS AND RESULTS

To validate the deployed models, a static test image of the digit "7" was embedded into the microcontroller code. The inference output was monitored via the STM32CubeIDE Live Expressions interface.

4.1. Accuracy & Model Saturation

All successfully deployed models (Custom CNN, SqueezeNet, MobileNetV2, ShuffleNet) correctly classified the test image with a confidence score of approximately 0.9999.



The screenshot shows the STM32CubeIDE Live Expressions window. The window has a toolbar with icons for Variables, Breakpoints, Expressions, Disassembly, Registers, Live Expressions, and SFRs. Below the toolbar is a table with three columns: Expression, Type, and Value. The table contains the following data:

Expression	Type	Value
<code>(x)= predicted_digit</code>	int	-1
<code>(x)= max_probability</code>	float	0
<code>ai_output_data</code>	float [10]	[10]
<code>(x)= ai_output_data[0]</code>	float	0
<code>(x)= ai_output_data[1]</code>	float	0
<code>(x)= ai_output_data[2]</code>	float	0
<code>(x)= ai_output_data[3]</code>	float	0
<code>(x)= ai_output_data[4]</code>	float	0
<code>(x)= ai_output_data[5]</code>	float	0
<code>(x)= ai_output_data[6]</code>	float	0
<code>(x)= ai_output_data[7]</code>	float	0
<code>(x)= ai_output_data[8]</code>	float	0
<code>(x)= ai_output_data[9]</code>	float	0
<code>+ Add new expression</code>		

Figure 1: The values of the variables before starting the process.

Expression	Type	Value
(x)= predicted_digit	int	7
(x)= max_probability	float	0.999930382
ai_output_data	float [10]	[10]
(x)= ai_output_data[0]	float	1.66101017e-08
(x)= ai_output_data[1]	float	2.75962047e-05
(x)= ai_output_data[2]	float	3.48327262e-06
(x)= ai_output_data[3]	float	1.61835345e-09
(x)= ai_output_data[4]	float	1.40071315e-08
(x)= ai_output_data[5]	float	2.47948577e-08
(x)= ai_output_data[6]	float	1.30893872e-13
(x)= ai_output_data[7]	float	0.999930382
(x)= ai_output_data[8]	float	5.42393908e-10
(x)= ai_output_data[9]	float	3.85087806e-05
+	Add new expression	

Figure 2: Real-time inference result on STM32F446RE showing the predicted digit (7) and high probability (0.9999).

Analysis: This uniform high performance is attributed to "Model Saturation." The MNIST dataset is relatively simple compared to the capabilities of modern architectures like Mobile Net or Shuffle Net, which are designed for ImageNet (1000 classes). Consequently, these models easily overperform on MNIST, converging to near-perfect confidence levels.

4.2. Hardware Resource Utilization

The following qualitative analysis summarizes the resource footprint observed during the build process:

Model	Flash Usage	RAM Usage	Feasibility on F446RE
Costom CNN	Very low	Low	Excellent
SqueezeNet	Medium	Low	Good
MobileNetV2	Medium - High	Medium	Feasible (Optimized)
ShuffleNet	Medium	Medium	Feasible
EfficientNet	Overflow	Overflow	Failed (Requires External memory)

As shown in the table, optimization techniques (such as reducing in MobileNet) were crucial for fitting advanced models into the microcontroller's internal Flash.

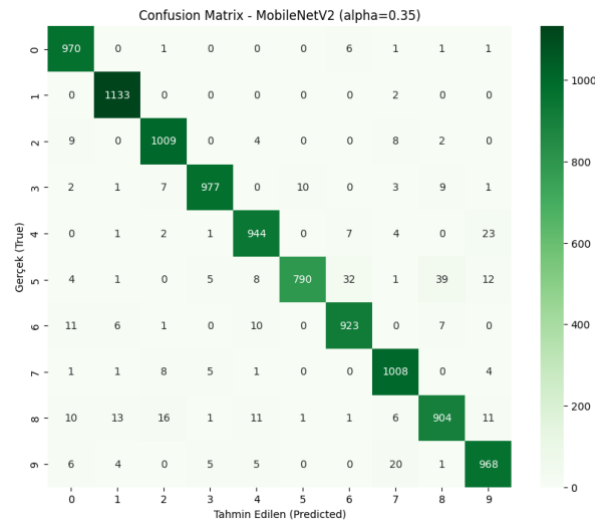


Figure 3: Confusion Matrices generated during Python training, verifying model performance before deployment.

5. CONCLUSION

This homework demonstrated the end-to-end deployment of Deep Learning models on an embedded microcontroller.

We successfully integrated SqueezeNet, MobileNetV2, and ShuffleNet on the STM32F446RE using the ST Edge AI toolchain.

The study highlighted that:

1. Architecture Matters: Specialized architectures like MobileNet and ShuffleNet provide a balance between accuracy and efficiency, but require specific tuning (e.g., Alpha scaling) for restricted hardware.
2. Hardware Constraints: Memory (Flash/SRAM) is the primary bottleneck in Edge AI. Models like EfficientNet, while powerful, are unsuitable for the F446RE without external memory.
3. Toolchain Efficiency: X-CUBE-AI's "RAM Optimization" and static buffer allocation significantly reduced the RAM footprint, enabling the deployment of relatively deep networks on a device with only 128 KB SRAM.

The complete source codes, trained model files (`.tflite`), and implementation details have been uploaded to the GitHub repository.

References

1. Ünsalan, C., Höke, B., & Atmaca, E. (2025). Embedded Machine Learning with Microcontrollers: Applications on STM32 Boards. Springer Nature.